

一、漫游

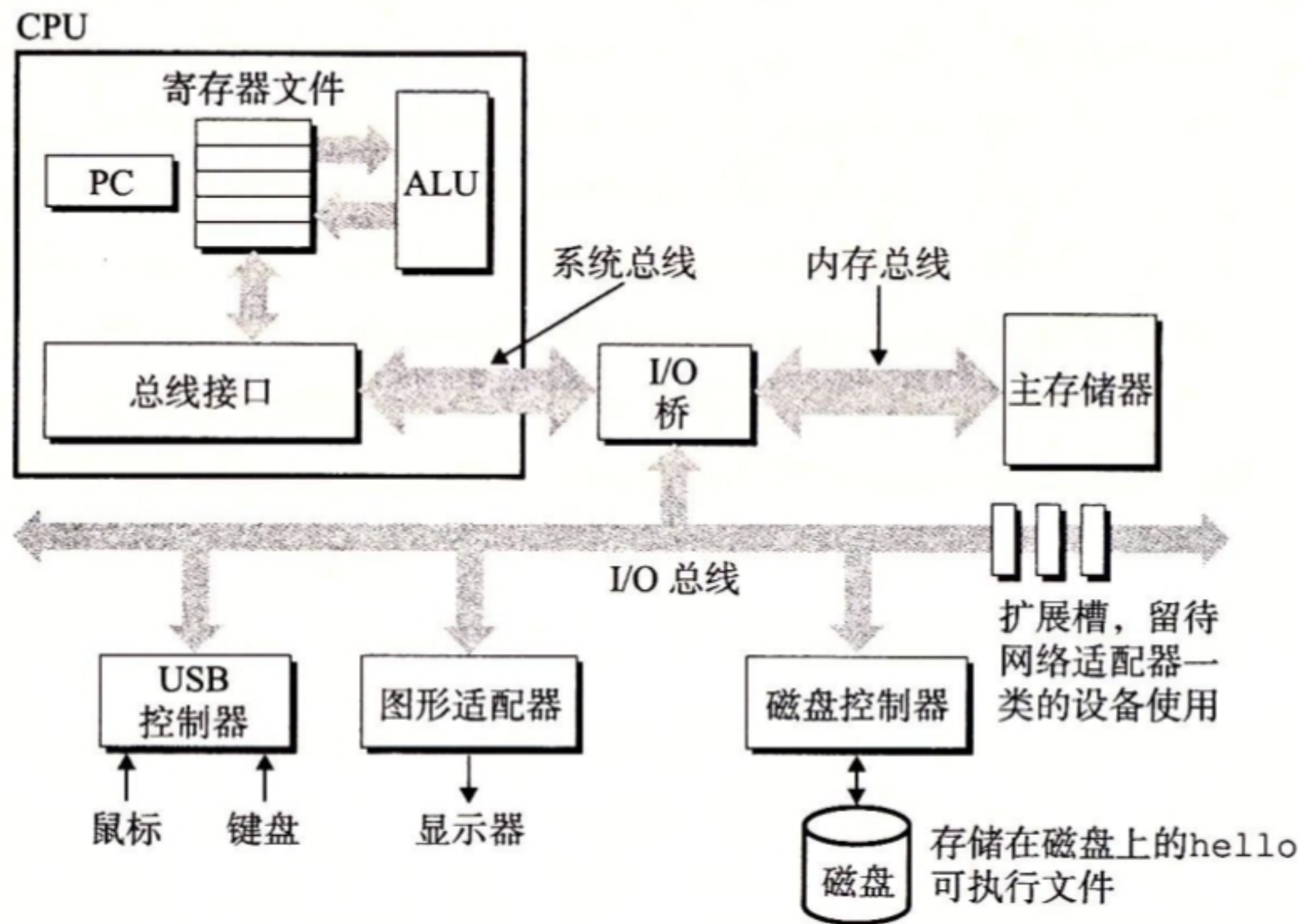
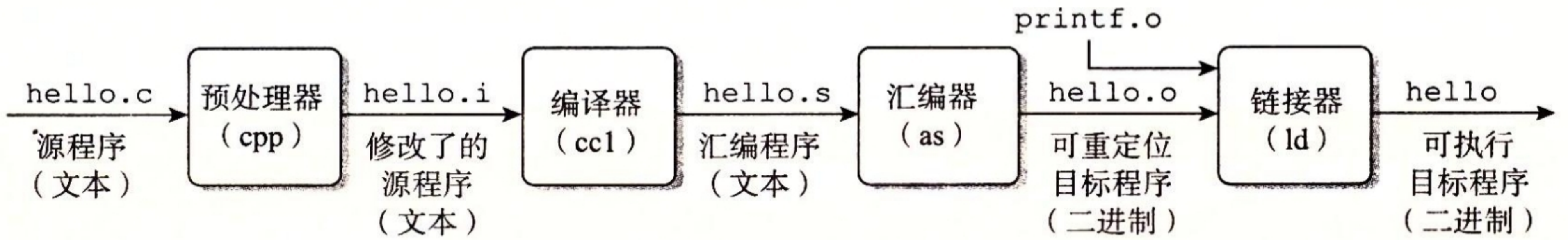


图 1-4 一个典型系统的硬件组成

CPU: 中央处理单元; ALU: 算术/逻辑单元; PC: 程序计数器; USB: 通用串行总线

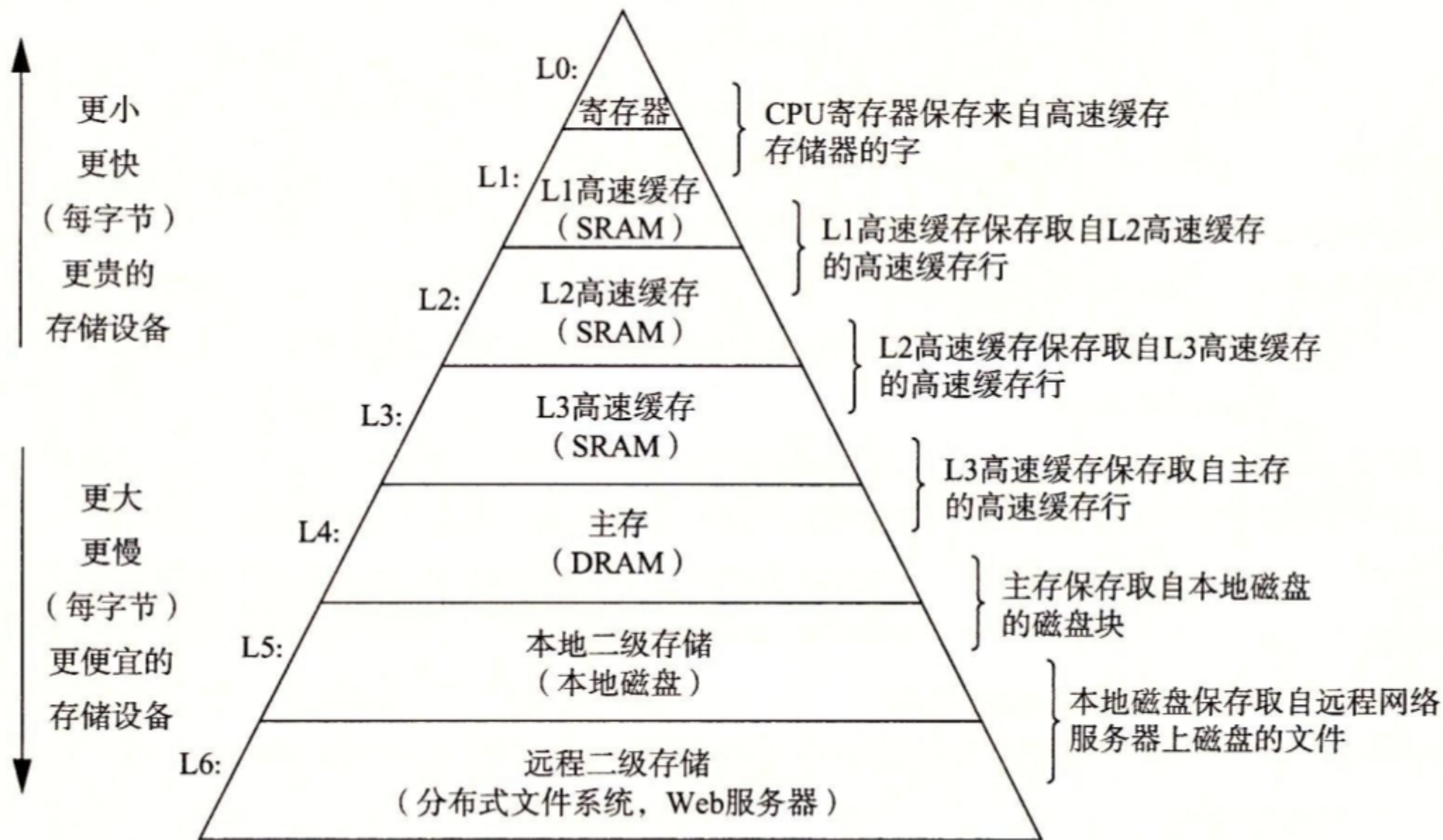


图 1-9 一个存储器层次结构的示例

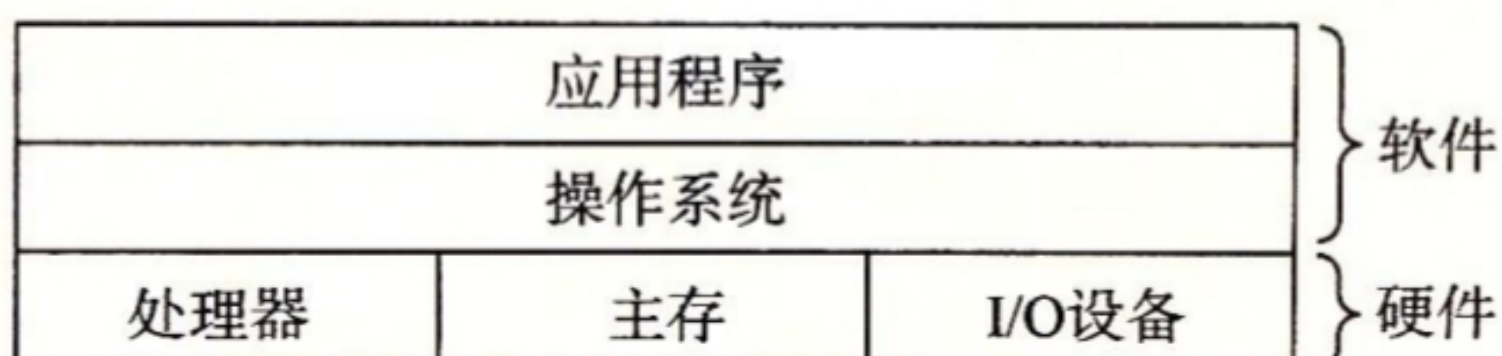


图 1-10 计算机系统的分层视图

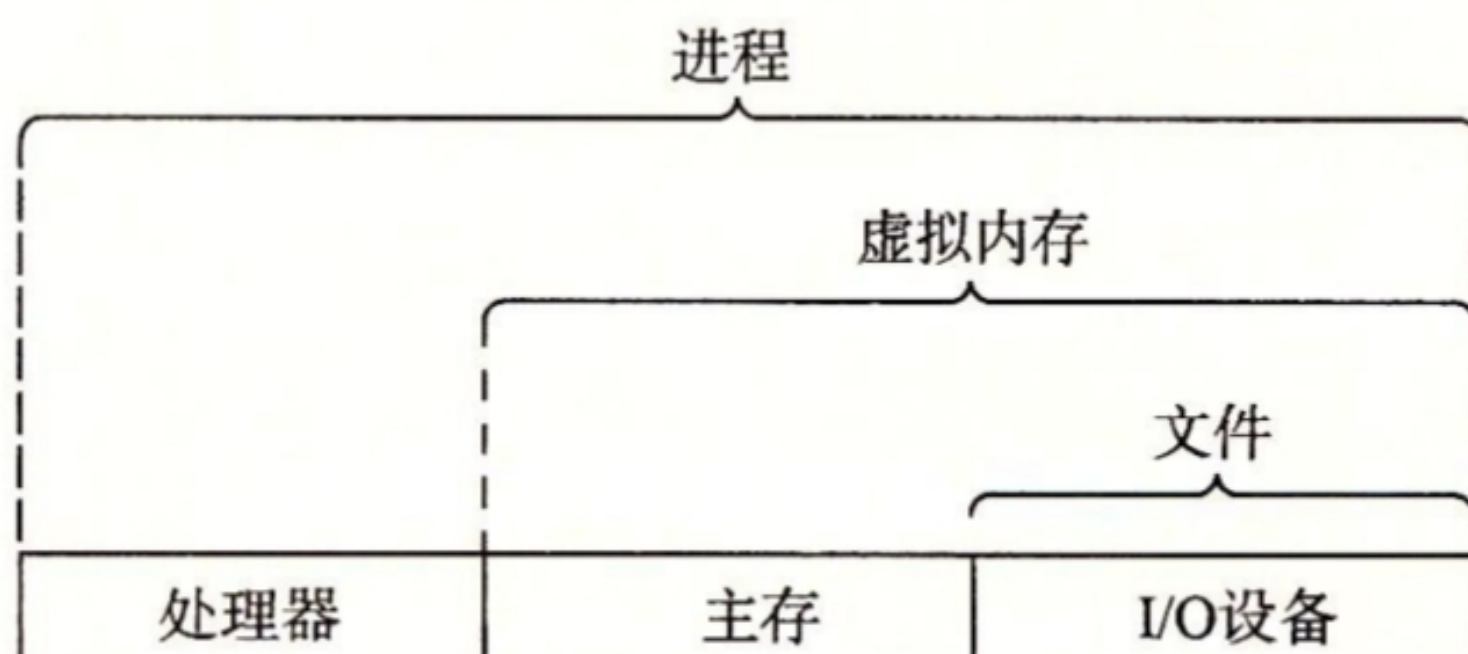


图 1-11 操作系统提供的抽象表示

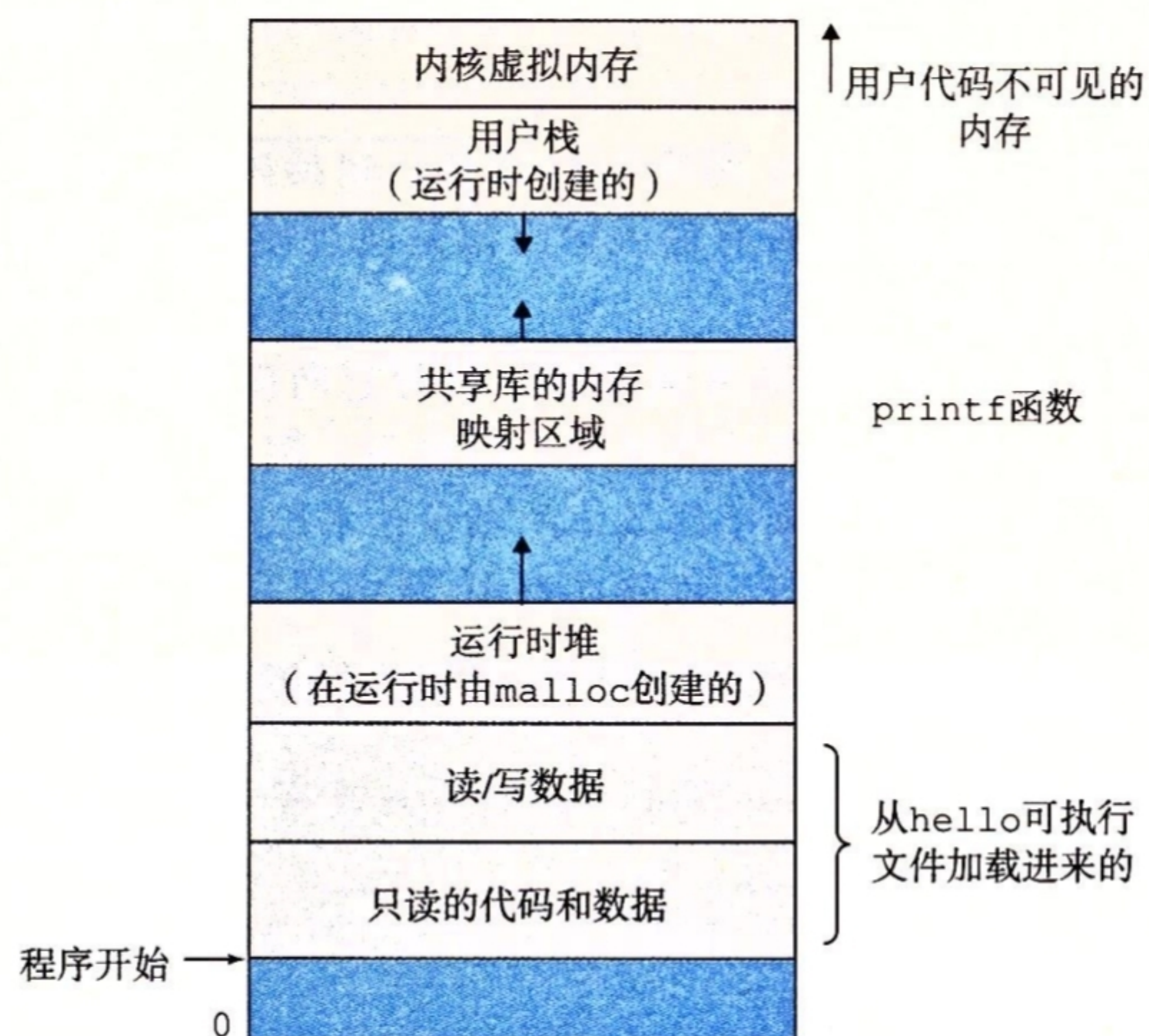


图 1-13 进程的虚拟地址空间

Amdahl 定律

$$T_{\text{new}} = (1 - \alpha) T_{\text{old}} + (\alpha T_{\text{old}}) / k = T_{\text{old}} [(1 - \alpha) + \alpha / k]$$

由此，可以计算加速比 $S = T_{\text{old}} / T_{\text{new}}$ 为

$$S = \frac{1}{(1 - \alpha) + \alpha / k} \quad (1.)$$

加速方式：并发和并行

- 线程级并发
- 指令级并行
- 单指令多数据并行

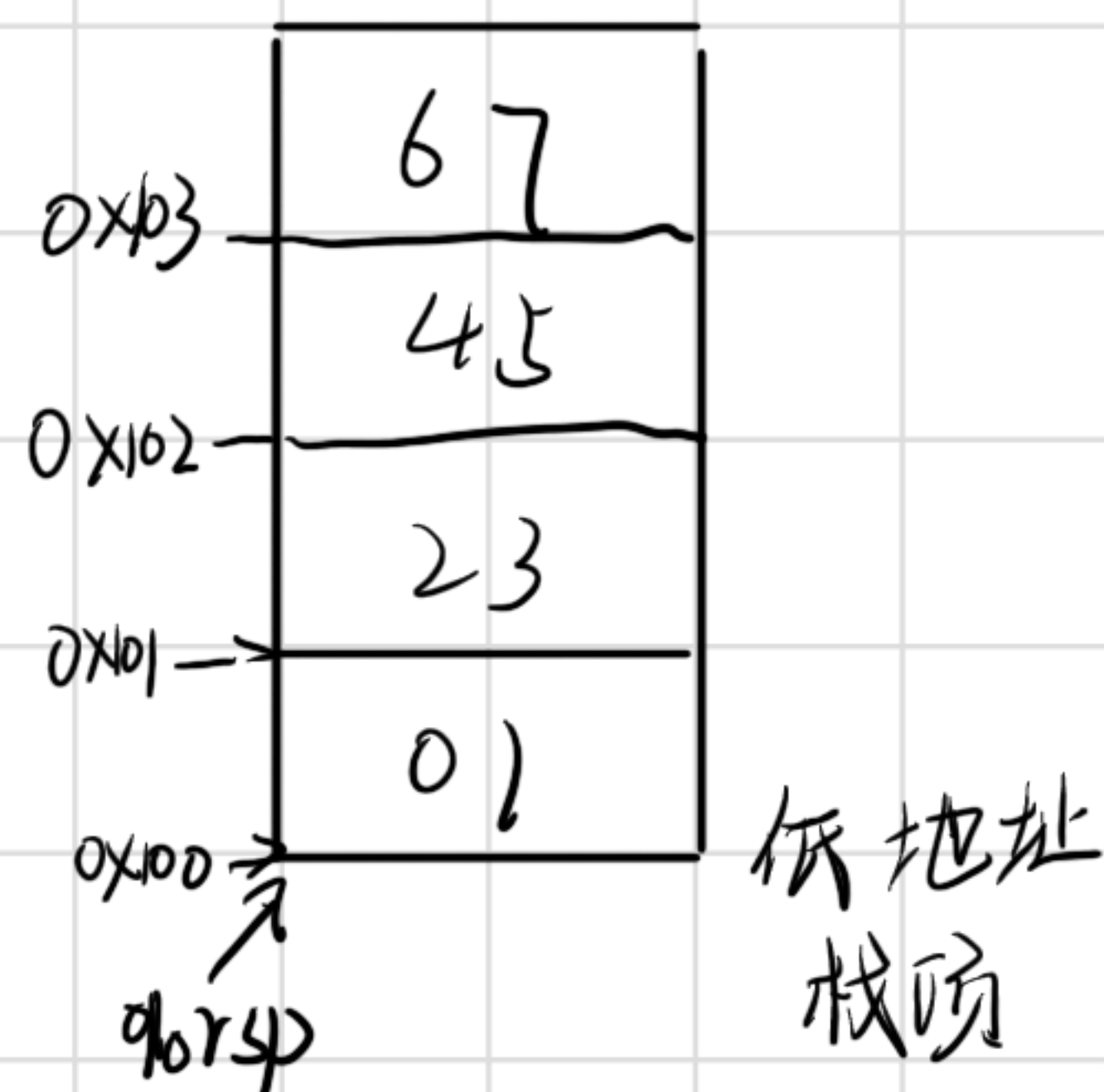
二、信息

1. 信息存储
2. 整数表示
3. 整数运算
4. 浮点数

1. 存储

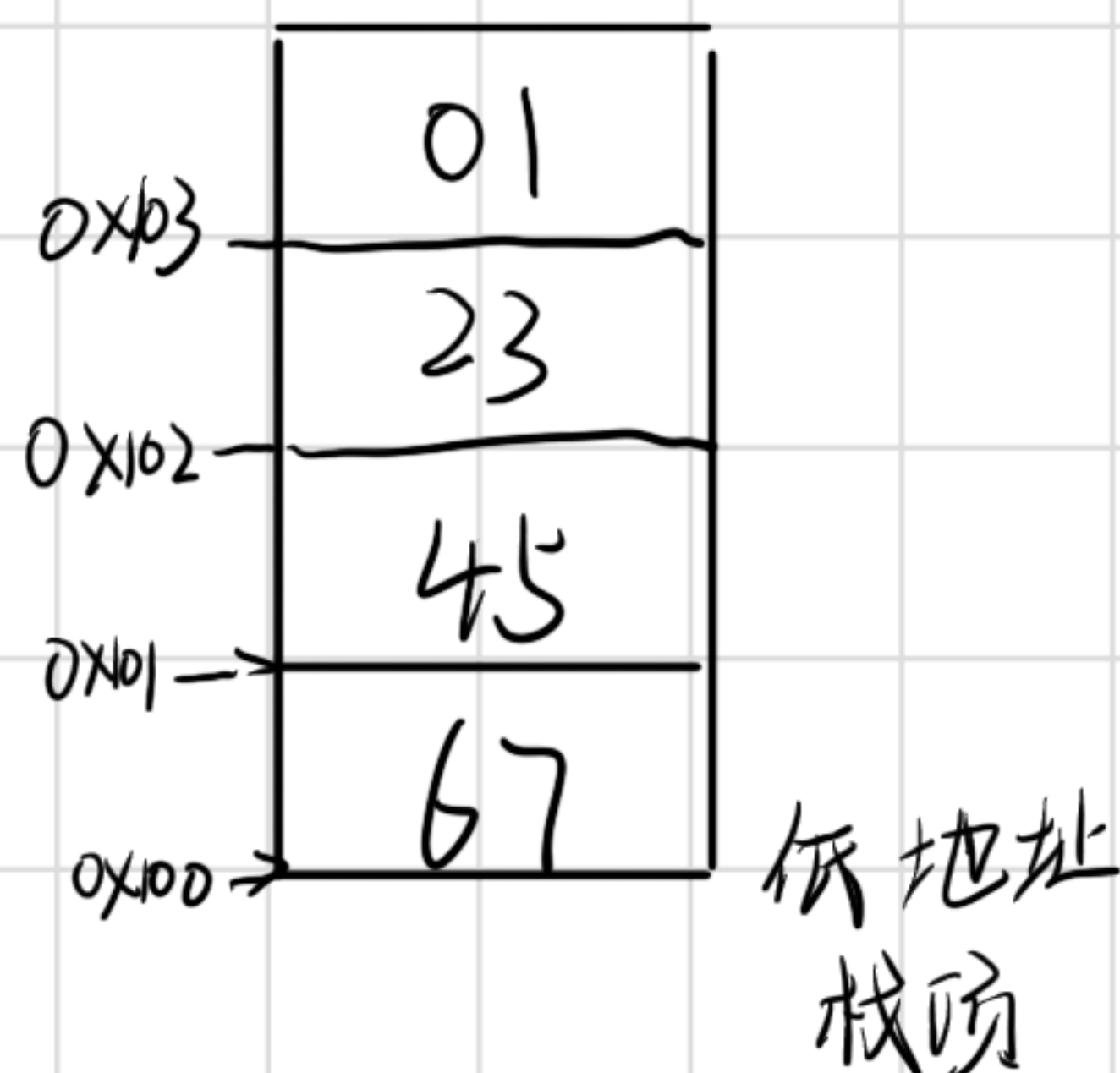
大端法: 0x1234567

0x100	0x101	0x102	0x103
01	23	45	67



小端法: 0x1234567

0x100	0x101	0x102	0x103
67	45	23	01



逻辑右移与算术右移

Operation(操作)	Value 1	Value 2
x	[01100011]	[10010101]
x<<4	[00110000]	[01010000]
x>>4(logical)	[00000110]	[00001001]
x>>4(arithmetic)	[00000110]	[11111001]

2. 整数表示

无符号数: 全为正

有符号数: 最高位为1时为负

补码 = 反码 + 1

强制转换: 位模式不变, 解释方式改变

隐式转换: 有符号数与无符号数运算, 有符号数转为无符号数

位数扩展: 无符号数补0 有符号数: 补最高位

位缩: 丢高位

3. 运算:

无符号数溢出: $x+y < x$ 和 y

有符号数溢出: 当 $x>0, y>0, x+y<0$ 发生正溢出
当 $x<0, y<0, x+y>0$ 发生负溢出

$x-y = x+y$ 的逆元

无符号数逆元: $y' = 2^n - y$

有符号数逆元: $y' = -y = y \text{ 反码} + 1$

乘:

用左移操作和加法操作替代

$$\begin{aligned} \text{eg. } x * 14 &= x * 8 + x * 4 + x * 2 \\ &= x \ll 3 + x \ll 2 + x \ll 1 \end{aligned}$$

除: 朝向 0 的方向舍入

无符号数: 逻辑右移

有符号数: 算数右移

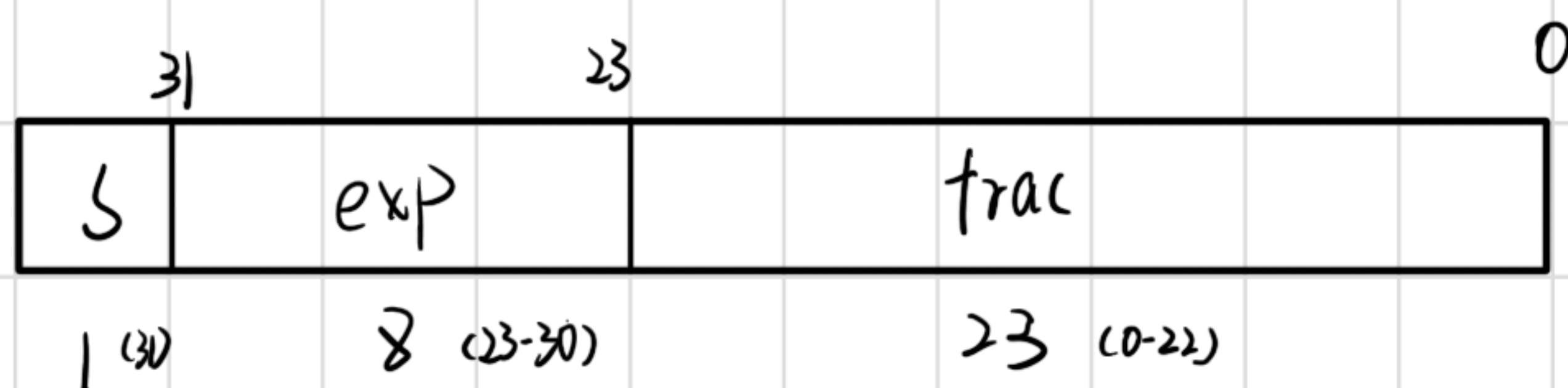
对于补码算术右移:

$x \geq 0$: 直接右移

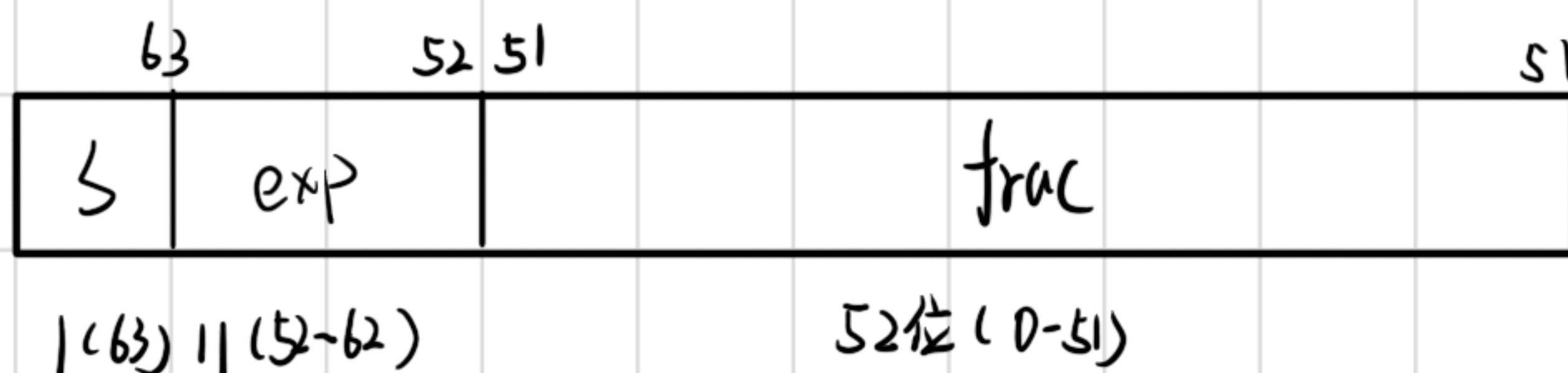
$x < 0$: 移 k 位前需加偏置量 $2^k - 1$

浮点数

float: 32位.



double: 64位



float:

1. 规格化的值

exp (阶码) 不全为0且不全为1 即 $\text{exp} \neq 0$ 且 $\text{exp} \neq 255$

$e_{\min}=1, e_{\max}=254$,

$\bar{E} = e - \text{偏置量}$ (float为127, double为1023)

$\bar{E}_{\min} = -126, \bar{E}_{\max} = 127$

$$M = 1.f_{22}f_{21}\dots f_1f_0 = 1+f$$

$$V = (-1)^s \times M \times 2^{\bar{E}}$$

2. 非规格化的值

exp全为0

$$M=f=0, s=0, V=+0.0$$

$$M=f=0, s=1, V=-0.0$$

$$M=f \neq 0,$$

$$V = (-1)^s f \times 2^e$$

3. 特殊值:

exp全为1

$$f=0, s=0, +\infty$$

$$f=0, s=1, -\infty$$

$$f \neq 0, \text{不是一个数}$$

e.g. $12,345 = 11\ 0000\ 0011\ 1001$ 二进制

$$= 1.1\ 0000\ 0011\ 1001 \times 2^{13}$$

↓

$$13 = e - \text{bias} = e - 127$$

$$100000011100100000000000; \text{frac}$$

$$e = 140 = 10001100 \text{ exp}$$

$$\Rightarrow 12,345_{\text{float}} = \underbrace{0}_{s} \underbrace{10001100}_{\text{exp}} \underbrace{100\ 0000\ 1110\ 0100\ 0000\ 0000}_{\text{frac}}$$

舍入：向偶数舍入

向零舍入 $1.5 \rightarrow 1$, $-1.5 \rightarrow -1$

向下舍入 $1.5 \rightarrow 1$, $-1.5 \rightarrow -2$

向上舍入 $1.5 \rightarrow 2$, $-1.5 \rightarrow -1$

向偶数舍入：当处于中间时的最低有效数位是偶数

$1.40 \rightarrow 1$ $1.50 \rightarrow 2$ $1.60 \rightarrow 2$ $2.50 \rightarrow 2$ $-1.50 \rightarrow 2$

四舍六入五凑偶

第三章

gcc -Og -S main.c $\Rightarrow$ main.s 汇编文件

gcc -Og -c main.c $\Rightarrow$ main.o 机器代码

objdump -d main.o $\Rightarrow$ main.s 反汇编代码

调用者保存寄存器 (主函数保存)

%rdi %rsi %rdx %rcx %rax %r8 %r9 %r10 %r11

被调用者保存寄存器 (子函数保存)

%rbx %rbp %r12 %r13 %r14 %r15

传参默认顺序: %rdi, %rsi, %rdx, %rcx, %r8, %r9

栈指针: %rsp

操作码

mov 源操作数 目的操作数
 立即数 地址(内存
 寄存器 寄存器
 地址(内存)

两个不能同时是地址

cltq = movslq, %eax, %rax (传低16位, 高16位 = 15位)

mov 7(%rdx, %rdx, 4), %rax: 将地址里的值传入

leaq 7(%rdx, %rdx, 4), %rax: %rax里是 $5\%rdx + 7$

指令	效果	描述
leaq S, D	$D \leftarrow \&S$	加载有效地址
INC D	$D \leftarrow D + 1$	加1
DEC D	$D \leftarrow D - 1$	减1
NEG D	$D \leftarrow -D$	取负
NOT D	$D \leftarrow \sim D$	取补
ADD S, D	$D \leftarrow D + S$	加
SUB S, D	$D \leftarrow D - S$	减
IMUL S, D	$D \leftarrow D * S$	乘
XOR S, D	$D \leftarrow D \sim S$	异或
OR S, D	$D \leftarrow D S$	或
AND S, D	$D \leftarrow D \& S$	与
SAL k, D	$D \leftarrow D \ll k$	左移
SHL k, D	$D \leftarrow D \ll k$	左移 (等同于SAL)
SAR k, D	$D \leftarrow D \gg_A k$	算术右移
SHR k, D	$D \leftarrow D \gg_L k$	逻辑右移

操作数

立即数: \$8

寄存器: %rax

内存引用: (%rdi)

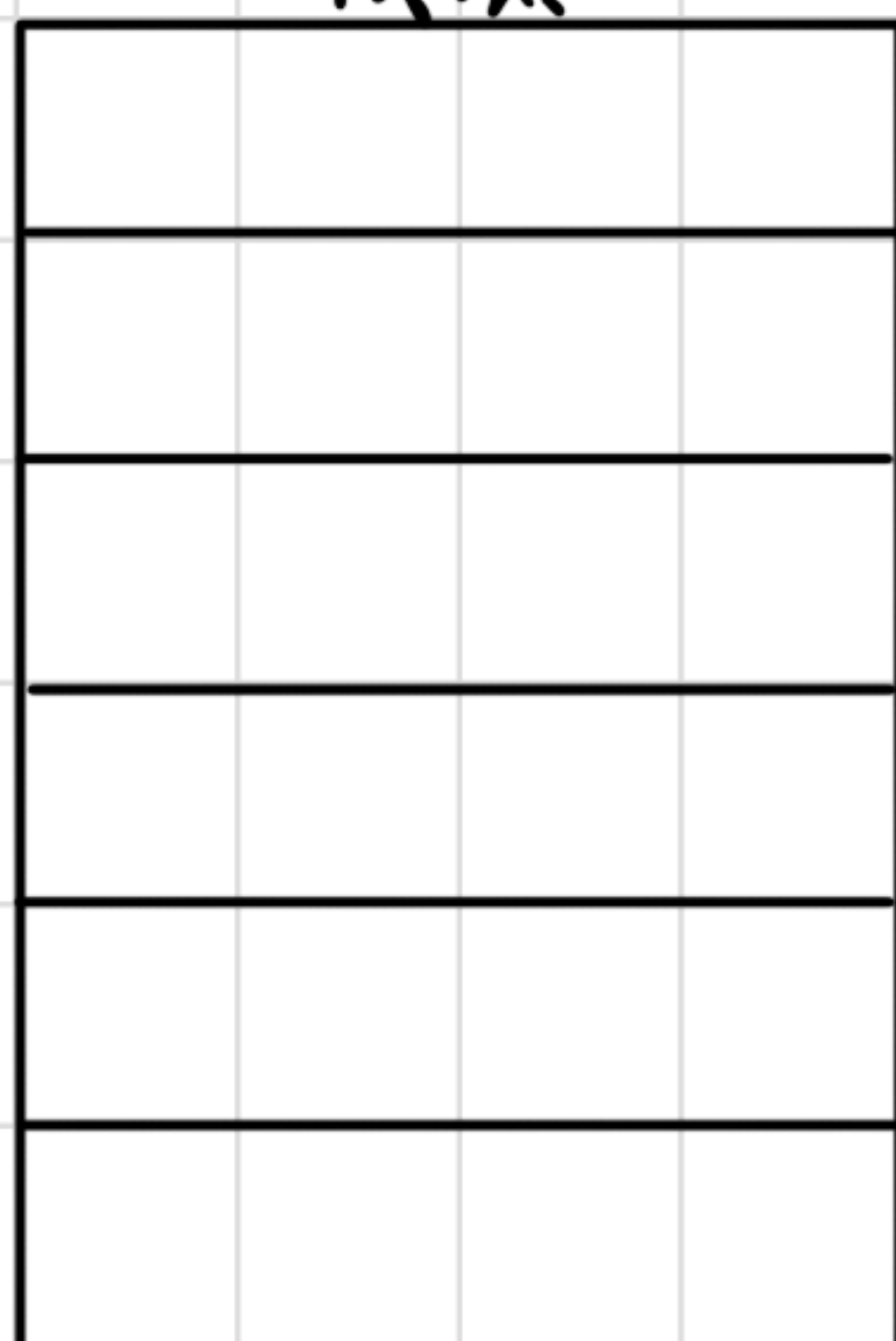
类型	格式	操作数值	名称
立即数	$\$Imm$	Imm	立即数寻址
寄存器	r_a	$R[r_a]$	寄存器寻址
存储器	Imm	$M[Imm]$	绝对寻址
存储器	(r_a)	$M[R[r_a]]$	间接寻址
存储器	$Imm(r_b)$	$M[Imm+R[r_b]]$	(基址 + 偏移量) 寻址
存储器	(r_b, r_i)	$M[R[r_b]+R[r_i]]$	变址寻址
存储器	$Imm(r_b, r_i)$	$M[Imm+R[r_b]+R[r_i]]$	变址寻址
存储器	$(, r_i, s)$	$M[R[r_i] \cdot s]$	比例变址寻址
存储器	$Imm(, r_i, s)$	$M[Imm+R[r_i] \cdot s]$	比例变址寻址
存储器	(r_b, r_i, s)	$M[R[r_b]+R[r_i] \cdot s]$	比例变址寻址
存储器	$Imm(r_b, r_i, s)$	$M[Imm+R[r_b]+R[r_i] \cdot s]$	比例变址寻址

图 3-3 操作数格式。操作数可以表示立即数(常数)值、寄存器值或是来自内存的值。比例因子 s 必须是 1、2、4 或者 8

栈

栈底

地址
减小
↓



→ 8字节

栈顶

push: 指针减8, 存入数据

pop: 读取数据, 指针加8

条件码

CF: 进位标志。最近的操作使最高位产生了进位。可用来检查无符号操作的溢出。

ZF: 零标志。最近的操作得出的结果为 0。

SF: 符号标志。最近的操作得到的结果为负数。

OF: 溢出标志。最近的操作导致一个补码溢出——正溢出或负溢出。

指令	同义名	效果	设置条件
sete <i>D</i>	setz	$D \leftarrow ZF$	相等/零
setne <i>D</i>	setnz	$D \leftarrow \sim ZF$	不等/非零
sets <i>D</i>		$D \leftarrow SF$	负数
setns <i>D</i>		$D \leftarrow \sim SF$	非负数
setg <i>D</i>	setnle	$D \leftarrow \sim (SF \wedge OF) \& \sim ZF$	大于 (有符号 >)
setge <i>D</i>	setnl	$D \leftarrow \sim (SF \wedge OF)$	大于等于 (有符号 >=)
setl <i>D</i>	setnge	$D \leftarrow SF \wedge OF$	小于 (有符号 <)
setle <i>D</i>	setng	$D \leftarrow (SF \wedge OF) \mid ZF$	小于等于 (有符号 <=)
seta <i>D</i>	setnbe	$D \leftarrow \sim CF \& \sim ZF$	超过 (无符号 >)
setae <i>D</i>	setnb	$D \leftarrow \sim CF$	超过或相等 (无符号 >=)
setb <i>D</i>	setnae	$D \leftarrow CF$	低于 (无符号 <)
setbe <i>D</i>	setna	$D \leftarrow CF \mid ZF$	低于或相等 (无符号 <=)

指令	同义名	跳转条件	描述
jmp <i>Label</i>		1	直接跳转
jmp <i>*Operand</i>		1	间接跳转
je <i>Label</i>	jz	ZF	相等/零
jne <i>Label</i>	jnz	$\sim ZF$	不相等/非零
js <i>Label</i>		SF	负数
jns <i>Label</i>		$\sim SF$	非负数
jg <i>Label</i>	jnle	$\sim (SF \wedge OF) \& \sim ZF$	大于 (有符号 >)
jge <i>Label</i>	jnl	$\sim (SF \wedge OF)$	大于或等于 (有符号 >=)
jl <i>Label</i>	jnge	$SF \wedge OF$	小于 (有符号 <)
jle <i>Label</i>	jng	$(SF \wedge OF) \mid ZF$	小于或等于 (有符号 <=)
ja <i>Label</i>	jnbe	$\sim CF \& \sim ZF$	超过 (无符号 >)
jae <i>Label</i>	jnb	$\sim CF$	超过或相等 (无符号 >=)
jb <i>Label</i>	jnae	CF	低于 (无符号 <)
jbe <i>Label</i>	jna	$CF \mid ZF$	低于或相等 (无符号 <=)

指令	同义名	传送条件	描述
<code>cmove S, R</code>	<code>cmovz</code>	ZF	相等/零
<code>cmovne S, R</code>	<code>cmovnz</code>	$\sim$ ZF	不相等/非零
<code>cmovs S, R</code>		SF	负数
<code>cmovns S, R</code>		$\sim$ SF	非负数
<code>cmovg S, R</code>	<code>cmovnle</code>	$\sim$ (SF $\wedge$ OF) & $\sim$ ZF	大于（有符号>）
<code>cmovge S, R</code>	<code>cmovnl</code>	$\sim$ (SF $\wedge$ OF)	大于或等于（有符号>=）
<code>cmovl S, R</code>	<code>cmovnge</code>	SF $\wedge$ OF	小于（有符号<）
<code>cmovle S, R</code>	<code>cmovng</code>	(SF $\wedge$ OF) ZF	小于或等于（有符号<=）
<code>cmova S, R</code>	<code>cmovnbe</code>	$\sim$ CF & $\sim$ ZF	超过（无符号>）
<code>cmovae S, R</code>	<code>cmovnb</code>	$\sim$ CF	超过或相等（无符号>=）
<code>cmovb S, R</code>	<code>cmovnae</code>	CF	低于（无符号<）
<code>cmovbe S, R</code>	<code>cmovna</code>	CF ZF	低于或相等（无符号<=）

```

if (test-expr)
    then-statement
else
    else-statement

```

这里 *test-expr* 是一个整数表达式。在两个分支语句中(*then-statement*和*else-statement*)，*test-expr* 的值非零时，执行 *then-statement*；否则，执行 *else-statement*。

对于这种通用形式，汇编控制流：

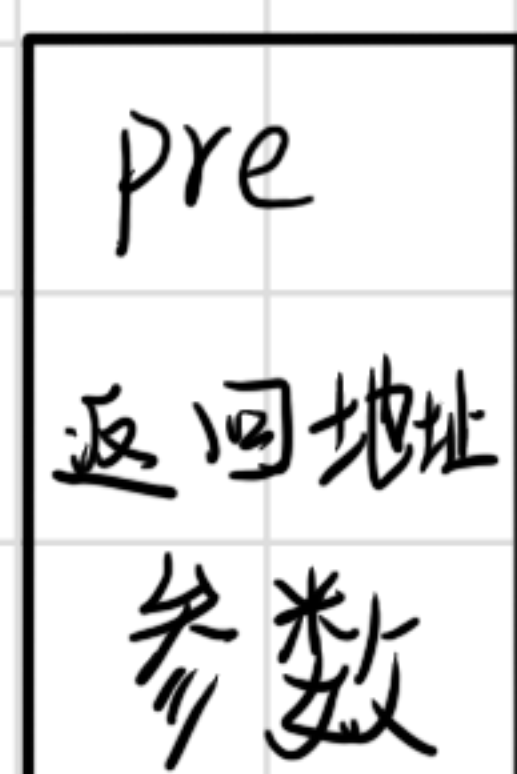
```

t = test-expr;
if (!t)
    goto false;
then-statement
goto done;
false:
    else-statement
done:

```


call 调用函数: 将 %rip 指向子函数
同时将返回地址压入栈中

当传参小于6个, 靠寄存器传递
当大于6个, 多出来的靠栈传递



结构体

对齐原则:任何 k 字节的数据类型起始地址必须为 k 的倍数

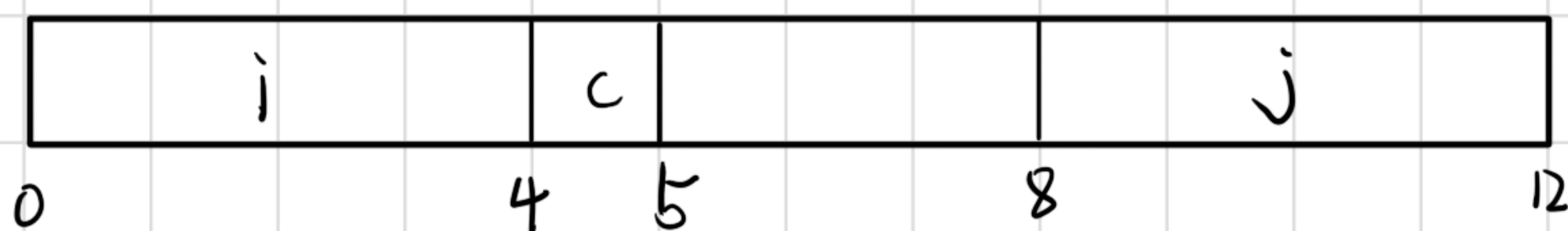
```
struct rec {
```

```
    int i;
```

```
    char c;
```

```
    int j;
```

```
}  ↓
```

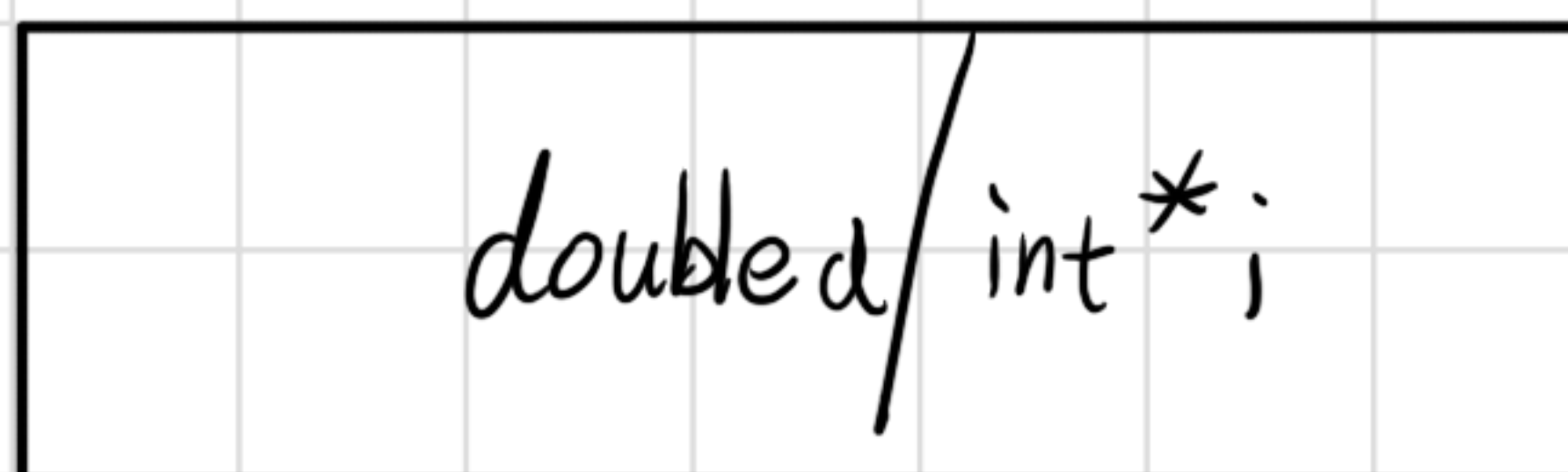


联合体

使用情况互斥时 使用联合体

```
union double d
```

```
    int * i
```



缓冲区溢出

栈随机化: 程序运行时会将地址随机分配与存储

金丝雀检测

pre
ret address
⋮
canary
buf

返回前检测金丝雀值是否被修改

消除向系统中插入可执行代码的能力
区分可读与可执行

第七章 链接

链接本质：合并相同的节

链接本质：合并相同的“节”

可重定位目标文件

系统代码	.text
系统数据	.data

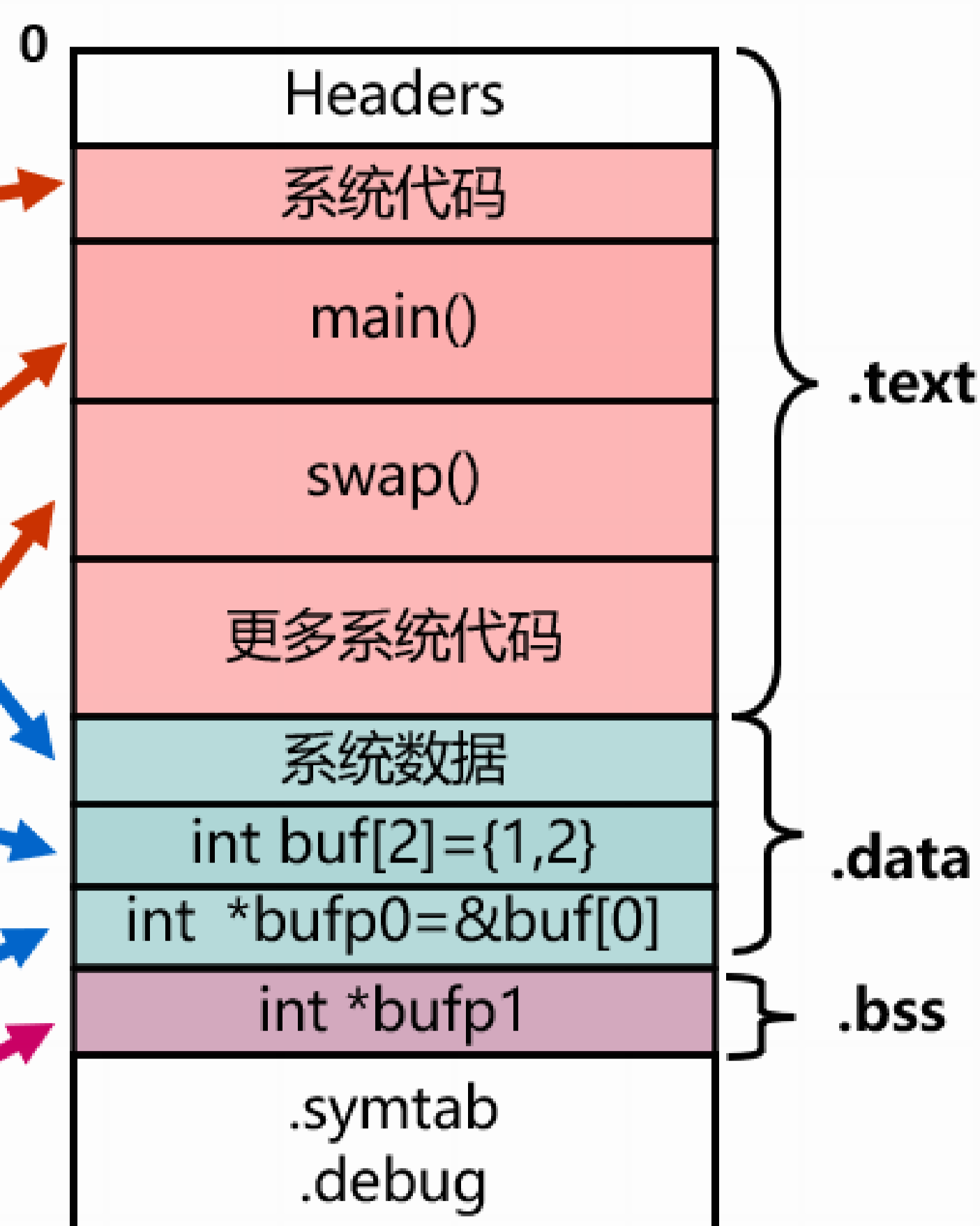
main.o

main()	.text
int buf[2]={1,2}	.data

swap.o

swap()	.text
int *bufp0=&buf[0]	.data
static int *bufp1	.bss

可执行目标文件



将可重定位目标文件和必要系统文件组合起来
生成可执行目标文件

可重定位目标文件

ELF header	Sections	Section header table
------------	----------	----------------------

ELF header 开头16字节:

7f	45	4c	46	02	01	01	00	00	00	00	00	00	00	00	00
DEL	E	L	F	↓	↓	↓									
				类型	字节序	标志									
ELF Magic				0x1:32位	0x1:小端										
ELF 魔数				0x2:64位	0x2:大端										

Section (节/段)

· data: 已初始化的全局变量和静态变量的值

· text:

Section header table: 记录每一段开头的位置

符号和符号表

全局符号: 该模块定义, 同时能被其它模块引用

func 函数

count 全局变量

value 全局变量

外部符号: 被其它模块定义, 同时被该模块引用

局部符号: 只能被该模块定义和引用

区别局部符号和全局变量: static 属性

带 static 属性的函数及变量不能被其他模块引用

ndx 列: section 索引值

符号

强符号: 函数和已初始化的全局变量

弱符号: 未初始化的全局变量

强-强重复: 报错

1强-多弱重复: 用强符号

多弱重复: 使用其中之一

使用静态库: `...a`

静态库即可重定位目标文件的集合

链接:

产生 3 个集合 E (已定义已使用)

U (未定义已使用)

D (已定义未使用)

新链接文件时搜索所有未定义已使用符号

链接自动最后链接 `libc.a` (无需写出)

全部链接完成后若 U 不空 $\Rightarrow$ 报错

若 U 空, 丢弃 D 将 E 整合

`foo.c` $\left\{ \begin{array}{l} \text{libz.a} \\ \text{libx.a} \end{array} \right\} \text{liby.a}$

z, x 要在 y 之前.

若相互引用可反复链接

重定位: 合并输入模块, 并为每个符号分配运行时地址

第一步: 重定位节和符号定义

将类型相同的section合并为一个新的section.

第二步: 对定义符号进行重定义(确定地址)

为函数和变量确认首地址

第三步: 重定位节中符号引用(确定地址)

对text和data节中每个符号引用修改地址

重定位条目的使用

PC相对地址:

运行时地址 = main起始地址 + 重定位条目中偏移量字段

$$ref_addr = ADDR(main) + r.offset$$

$$*ref_ptr = ADDR(sum) - ref_addr + r.addend$$

(求两个地址之间的相对位置) $\sum$ $\uparrow$ call后下一条指令地址 $\uparrow$

$$05\ 00\ 00\ 00 \rightarrow 0x5 = 4004e8 - 4004e3$$

eg.

4004de e8 00 00 00 00 callq 4004e8(sum)

4004e3 48 83 c4 08 add \$0x8,%rsp

4004e8 : sum

绝对地址：操作地址

```
bf 00 00 00 00 mov $0x0, %edi
```

array: 0x601018

```
⇒ bf 18 10 60 00 mov $0x601018, %edi
```

可执行文件：与可重定位目标文件类似。

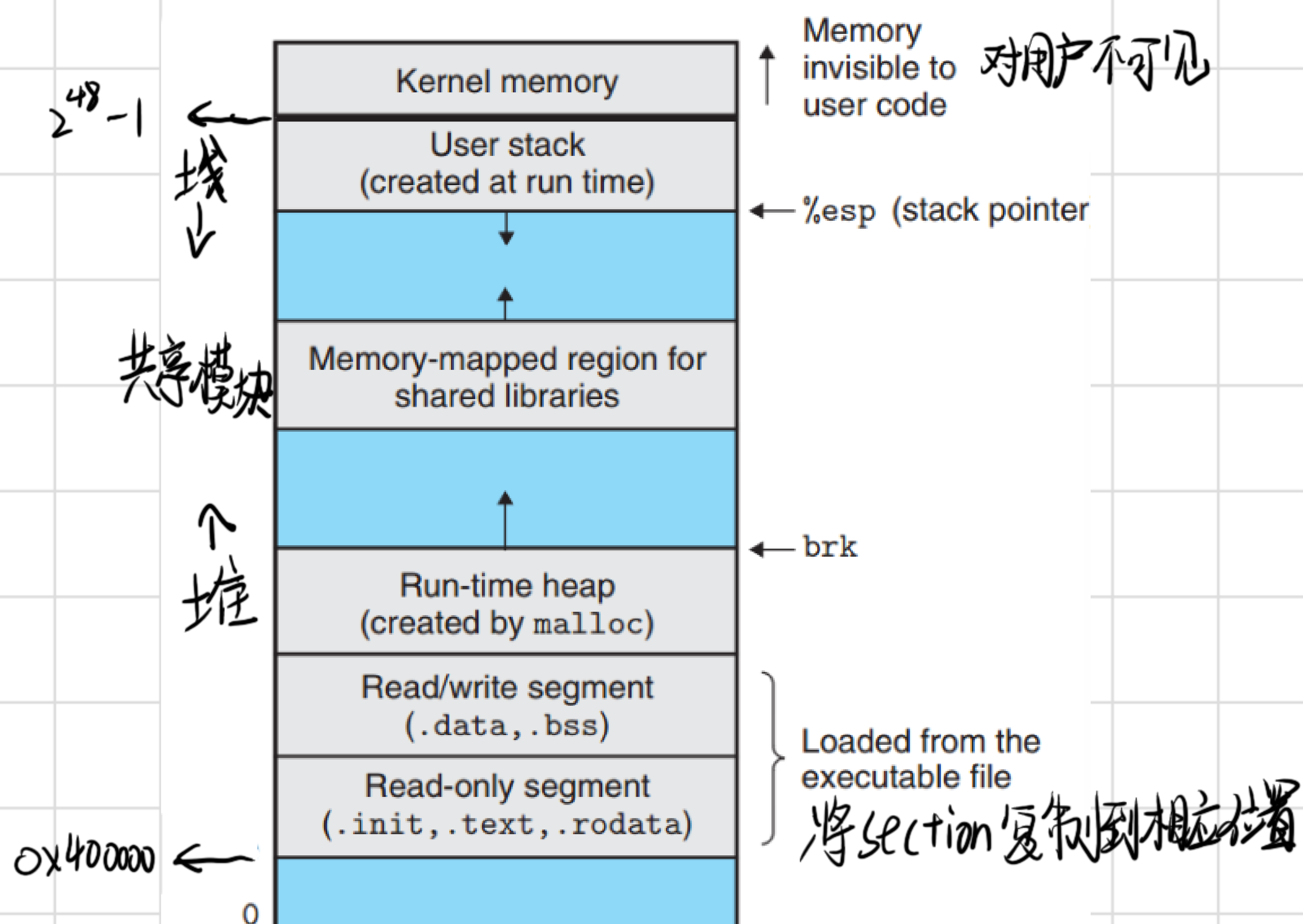
ELF Header 包含程序入口

多 .init section 初始化

少 .rel section

加载：将程序从磁盘复制到内存并运行

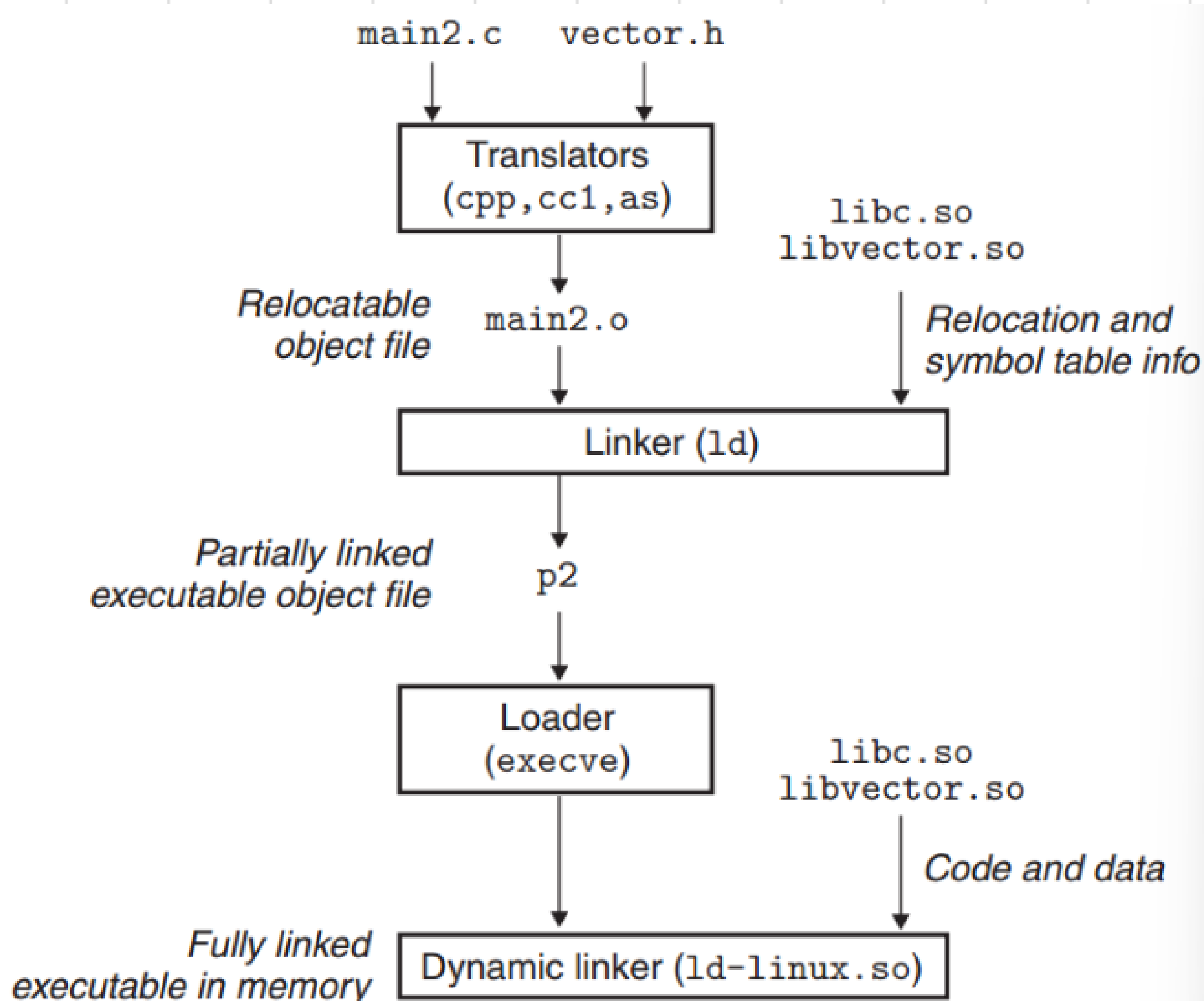
每个linux程序都有一个运行时内存镜像



动态库 (共享库)

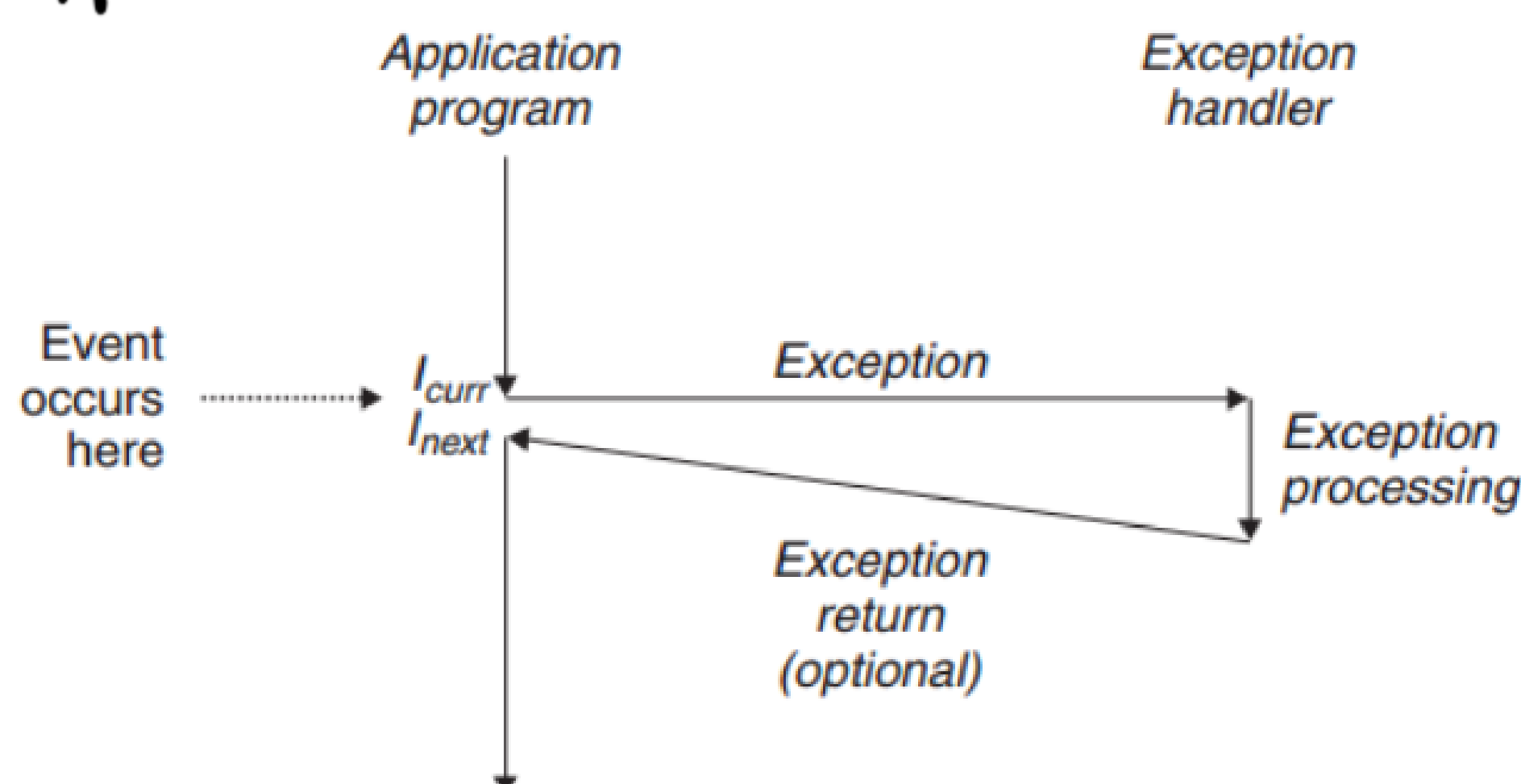
Linux: .so

Windows: .dll



第八章 异常控制流

异常

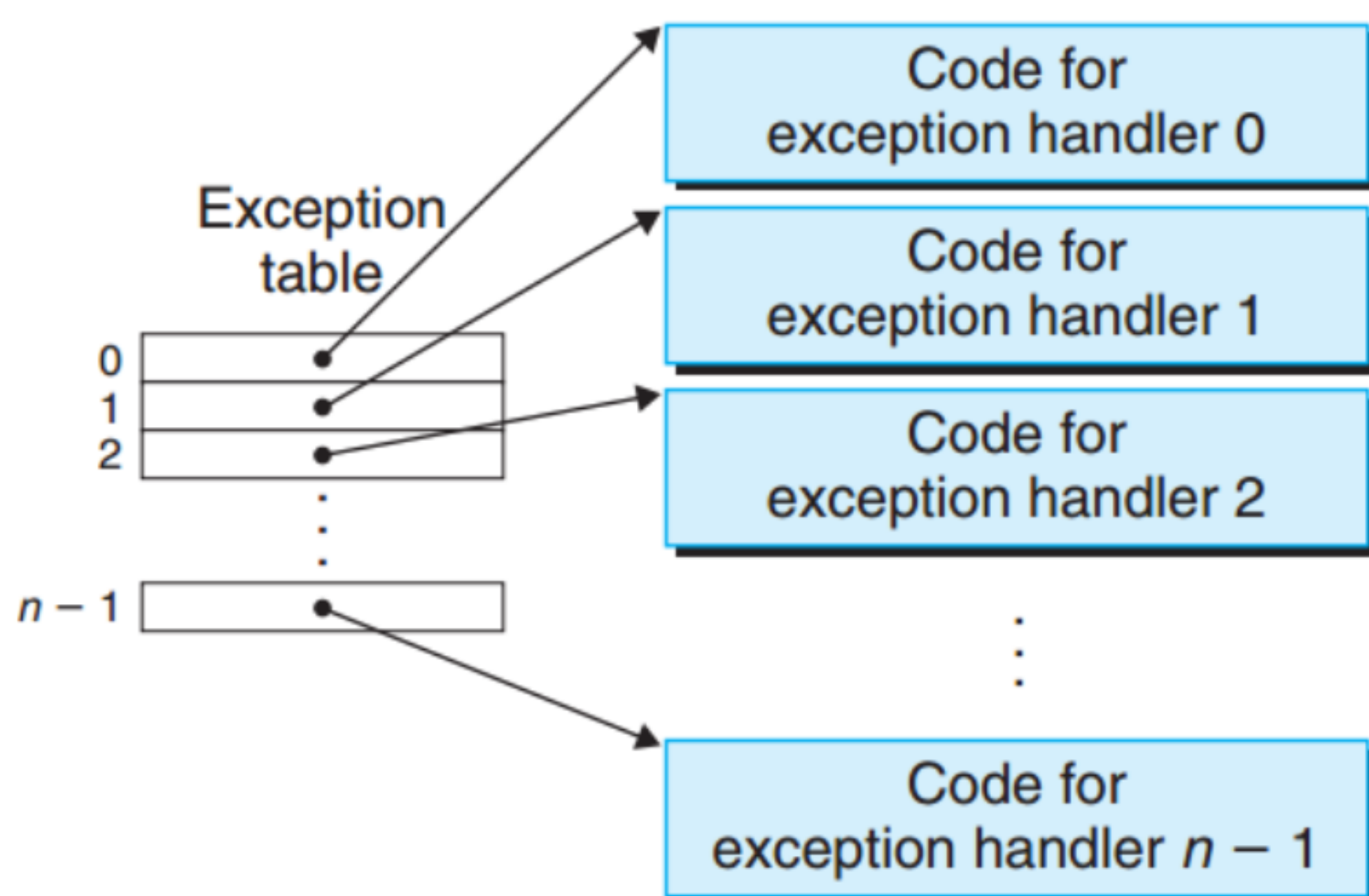


当处理器发现有事件发生时, 跳转到异常表
调用异常处理程序

每种异常都有一个唯一-非负整数的异常号

异常表起始地址在异常表基址寄存器

运行在内核态, 对所有系统资源都有访问权限



三种返回情况:

返回当前指令 I_{curr}

返回下一指令 I_{next}

返回中断程序的程序

四类异常：中断、陷阱、故障、中止

异步 同步

中断：由处理器外部的I/O设备信号的结果。
不由任何一条专门的指令造成
e.g. 键盘输入

陷阱：执行一条指令的结果，故意触发的异常
用途：为用户程序和内核之间提供了过程一样的接口
syscall 转到陷阱处理程序

故障：发生故障就转到故障修复程序
如果修复返回给指令
如果未修复返回 abort 例程终止程序

终止：通常是硬件错误，一般转至 abort

共255种异常，0-31号为架构师定义
32-255由操作系统定义

第十章 系统级I/O

I/O 输入/输出 是在主存 memory 和外部设备之间拷贝数据的过程

UNIX I/O

所有I/O都模型化为文件.

输入输出统一方式:

打开文件

改变当前文件位置

读写文件

关闭文件

打开和关闭文件

```
int fd;    /* file descriptor */

if ((fd = open("/etc/hosts", O_RDONLY)) < 0) {
    perror("open");
    exit(1);
}
```

```
int fd;    /* file descriptor */
int retval; /* return value */

if ((retval = close(fd)) < 0) {
    perror("close");
    exit(1);
}
```


读和写文件

```
#include <unistd.h>
```

```
ssize_t read(int fd, void *buf, size_t n);
```

返回：若成功则为读的字节数，若 EOF 则为 0，若出错为 -1。

```
ssize_t write(int fd, const void *buf, size_t n);
```

返回：若成功则为写的字节数，若出错则为 -1。

read: 从 fd 的当前文件位置拷贝至多 n 个字节到 buf

write: 从 buf 位置拷贝至多 n 个字节到 fd 的当前文件位置

lseek 函数: 显式地修改当前文件位置。

共享文件.

可以用许多不同的方式来共享 Unix 文件。内核用三个相关的数据结构来表示打开的文件：

- **描述符表**(descriptor table)。每个进程都有它独立的描述符表，它的表项是由进程打开的文件描述符来索引的。每个打开的描述符表指向文件表中的一个表项。
- **文件表**(file table)。打开文件的集合是由一张文件表来表示的，所有的进程共享这张表。每个文件表的表项包括当前的文件位置、引用计数(reference count)，以及一个指向 v-node 表中对应表项的指针。
- **v-node 表**(v-node table)。同文件表一样，所有的进程共享这张表。每个表项包含 stat 结构中的大多数信息。

示例1:
描述符1和描述符4通过不同的打开文件表表项来引用两个不同的文件。没有共享文件，并且每个描述符对应一个不同的文件。

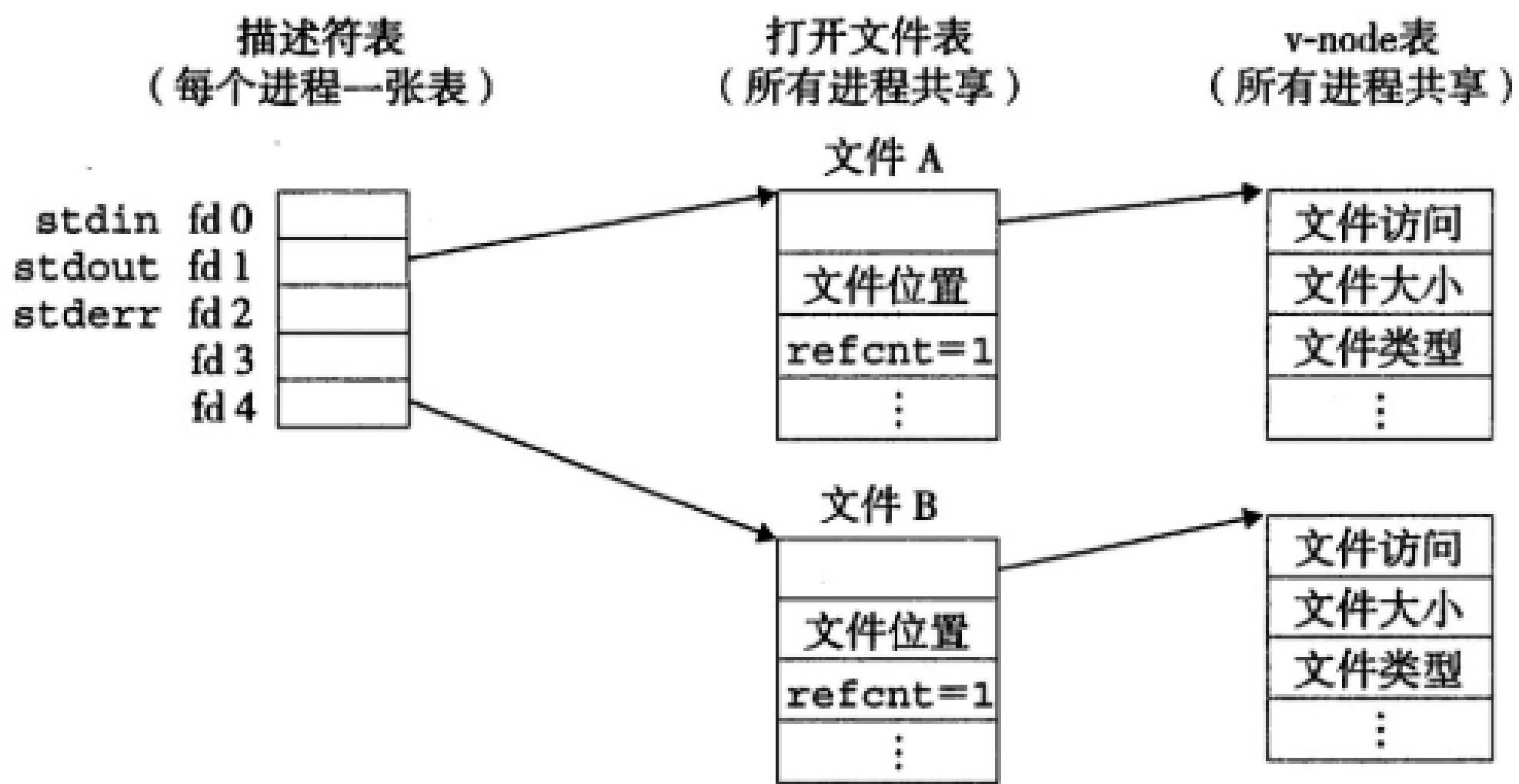


图 10-11 典型的打开文件的内核数据结构。在这个示例中，两个描述符引用不同的文件。没有共享

示例2:

多个文件描述符通过不同的文件表表项来引用同一个文件。
同一个filename调用open两次，关键思想是每个描述符都有它自己的文件位置，所以对不同的描述符的读操作可以从文件不同的位置获取数据。

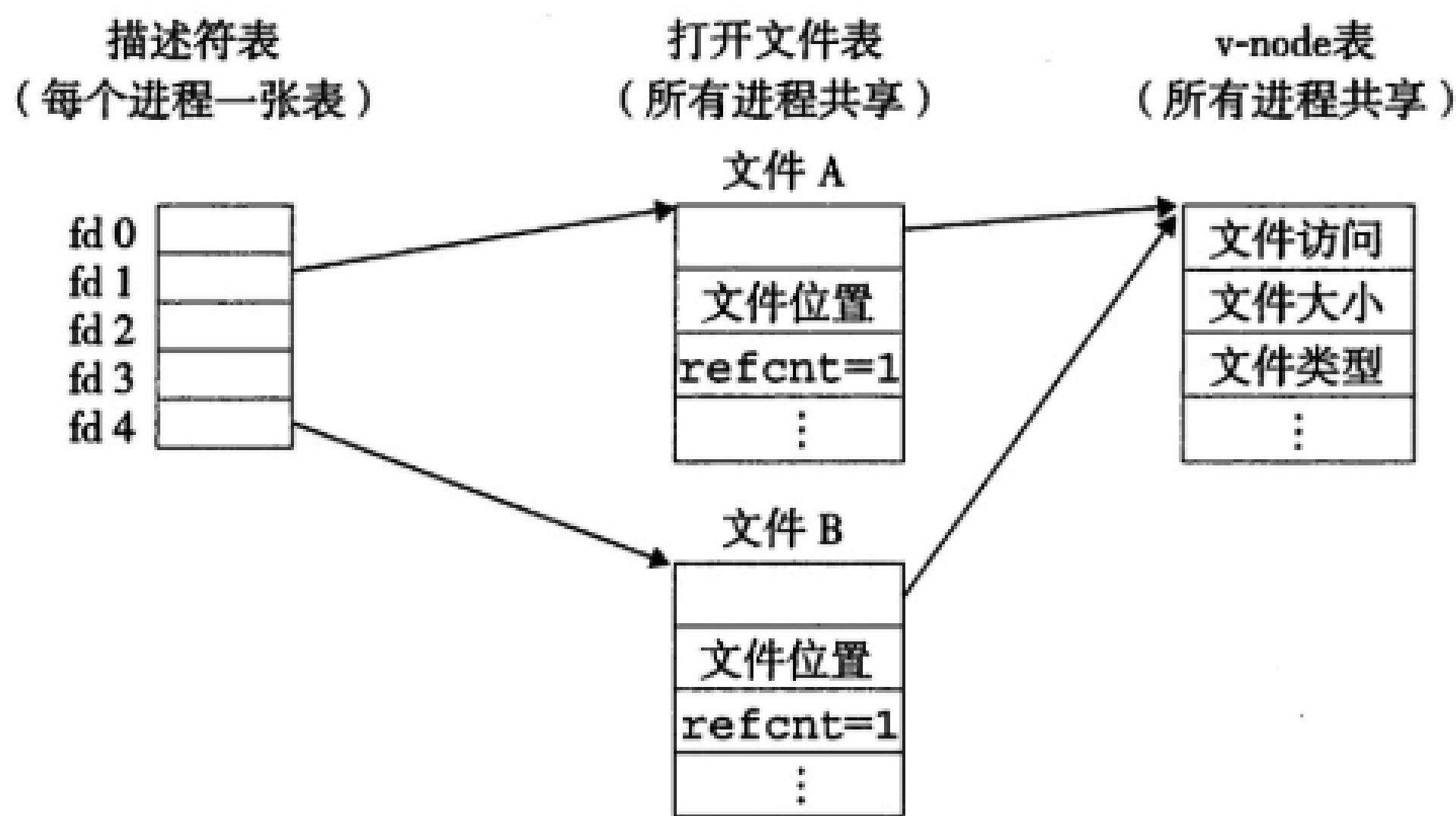


图 10-12 文件共享。这个例子展示了两个描述符通过两个打开文件表表项共享同一个磁盘文件

示例3:
父子进程共享文件。假设调用fork之前，父进程如图10-11所示的打开文件。图10-13展示了调用fork后的情况。子进程有一个父进程描述符的副本。父子进程共享相同的打开文件表集合，因此共享相同的文件位置。一个很重要的结果就是在内核删除相应的文件表表项之前，父子进程必须都关闭了它的描述符。

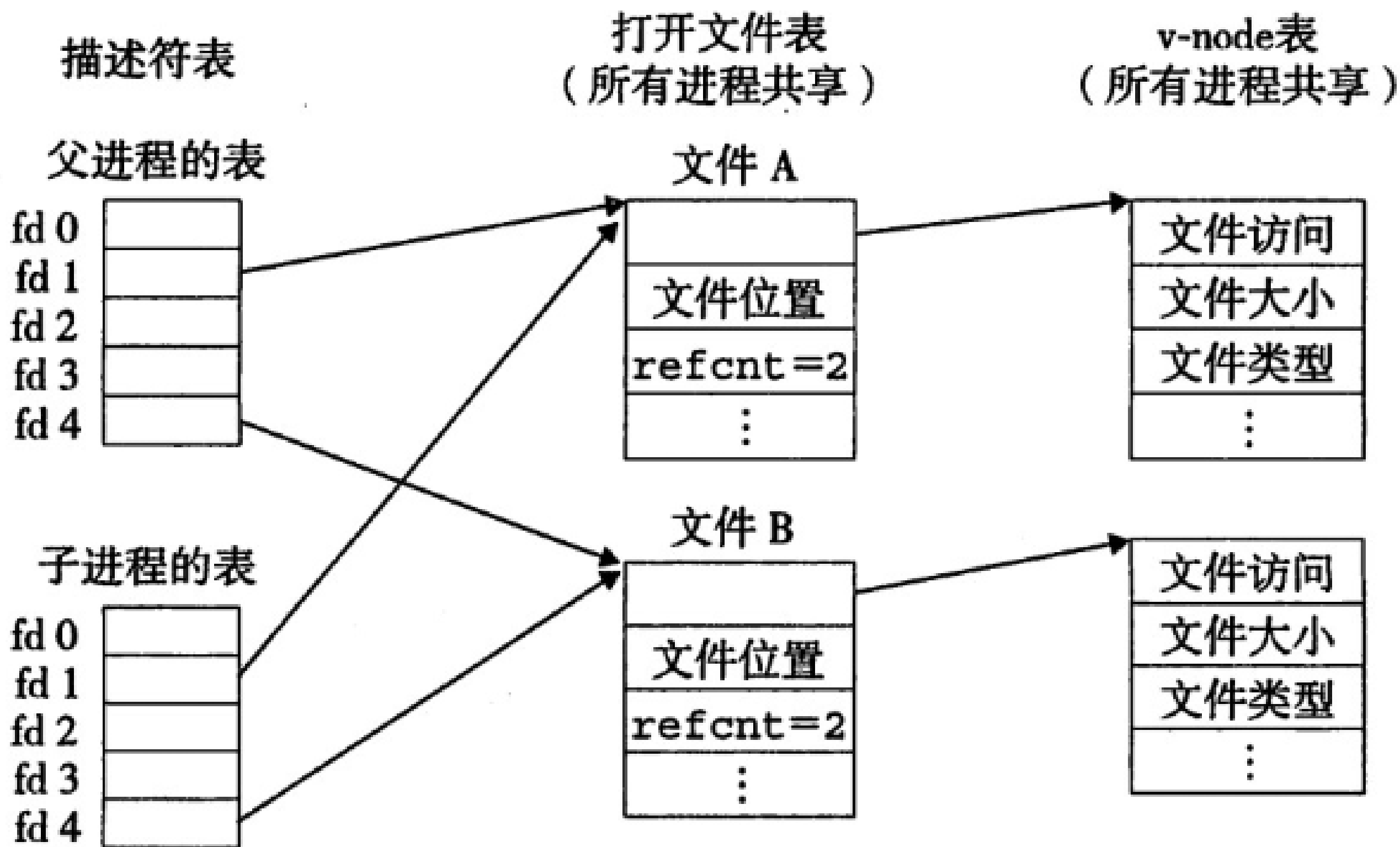
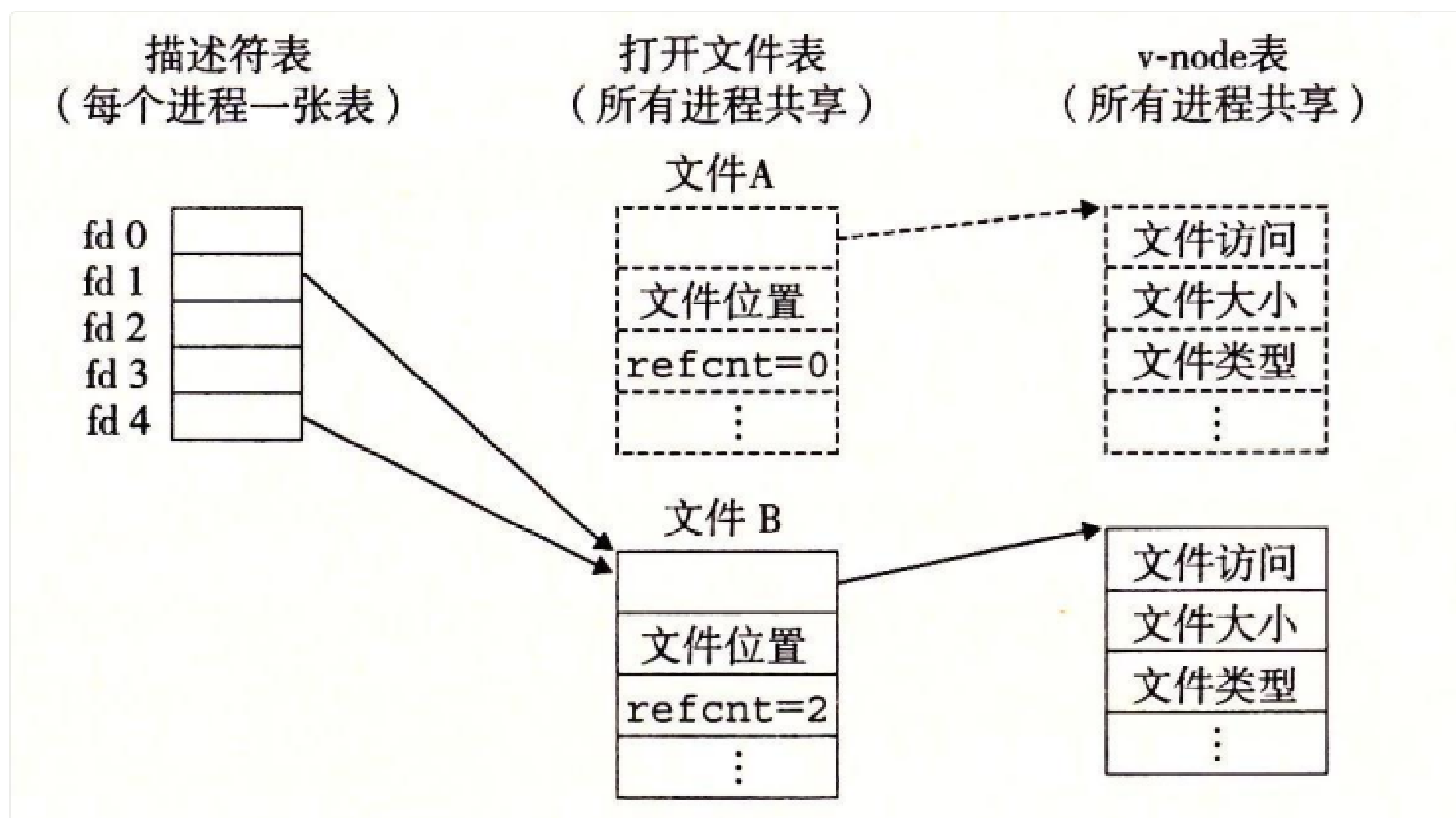


图 10-13 子进程如何继承父进程的打开文件。初始状态如图 10-11 所示

I/O 重定向

dup2 函数复制描述符表表项 oldfd 到描述符表表项 newfd，覆盖描述符表表项 newfd 以前的内容。如果 newfd 已经打开了，dup2 会在复制 oldfd 之前关闭 newfd。

假设在调用 **dup2(4,1)** 之前，我们的状态如图 10-12 所示，其中描述符 1（标准输出）对应于文件 A（比如一个终端），描述符 4 对应于文件 B（比如一个磁盘文件）。A 和 B 的引用计数都等于 1。图 10-15 显示了调用 **dup2(4,1)** 之后的情况。两个描述符现在都指向文件 B；文件 A 已经被关闭了，并且它的文件表和 v-node 表表项也已经被删除了；文件 B 的引用计数已经增加了。从此以后，任何写到标准输出的数据都被重定向到文件 B。



7. 可用的I/O

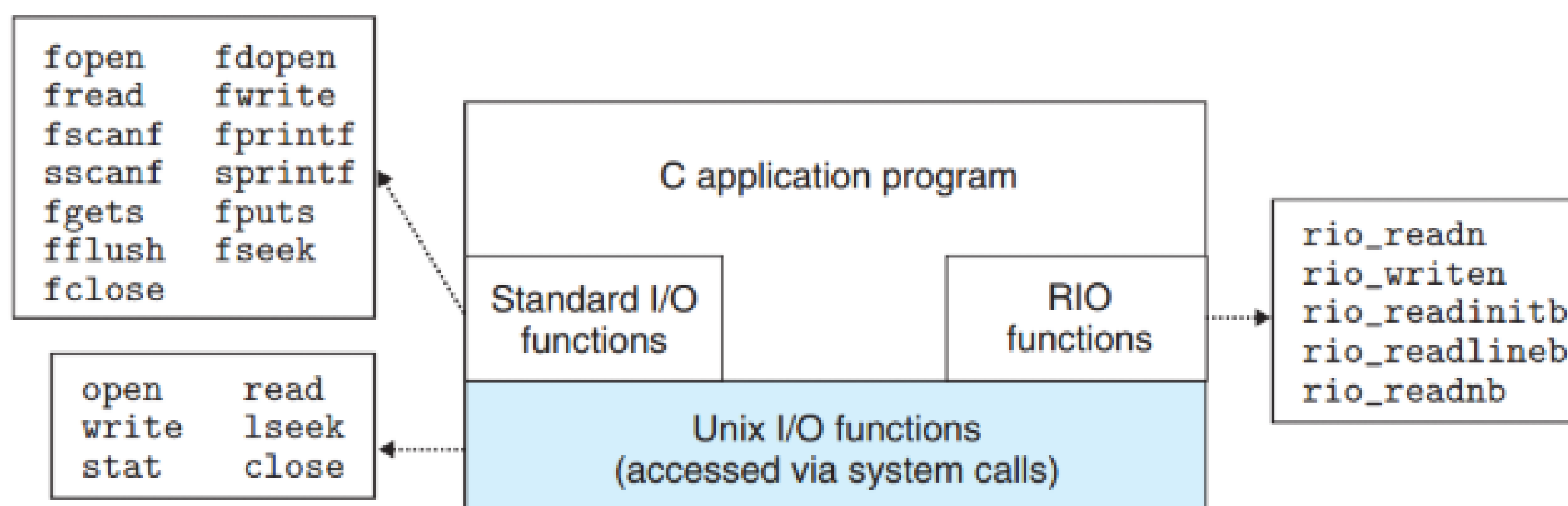


Figure 10.15 Relationship between Unix I/O, standard I/O, and RIO.